

UNIVERSIDAD PERUANA UNION
FACULTAD DE INGENIERÍA Y ARQUITECTURA
Escuela Profesional de Ingeniería de Sistemas



Entregable 2: Plataforma de Datos del Sistema (Alineado con CE022)

Línea de Investigación	CE02: Ingeniería de Software
Semestre	2026-I
Tipo	CE022 - Entregable 2: Plataforma de Datos del Sistema
Estudiante	Condori Pachauri Omar Flores Llanque Mery Elizabeth Huahuaccapa Ccama Jaqueline
Código	201810211 202211953 202210712

JULIACA, PERU

2026

Contenido

Entregable 2: Plataforma de Datos del Sistema (Alineado con CE022)	1
RESUMEN EJECUTIVO	3
SECCIÓN 1: MODELO DE DATOS Y MAPEO DE CAPTURAS	3
1.1 Módulo de Seguridad, Accesos y Menú Dinámico	3
1.2 Módulo de Catálogo General, Productos y Kardex Logístico	6
1.3 Módulo de Tesorería, Cajas Chicas y Caja General	10
1.4 Módulo de Logística y Compras a Proveedores.....	16
1.5 Módulo de Cuentas Corrientes, Convenios y Paquetes	19
1.6 Módulo de Facturación y Consolidación de Sucursal.....	24
1.7 Justificación de la Normalización de Base de Datos	26
SECCIÓN 2: IMPLEMENTACIÓN DE BASE DE DATOS (SCRIPTS DDL & DML)	27
2.1 Scripts DDL (PostgreSQL)	27
2.2 Scripts DML (Carga Semilla)	29
SECCIÓN 3: CONSULTAS Y PROGRAMACIÓN (TRIGGERS & PROCEDURES)....	30
3.1 Triggers Transaccionales en PL/pgSQL.....	30
3.2 Funciones y Procedimientos Almacenados (PL/pgSQL)	31
3.3 Consultas Analíticas y de Negocio	32
3.4 Evidencias de Optimización (Índices & EXPLAIN ANALYZE)	33
SECCIÓN 4: SEGURIDAD, AUDITORÍA Y ADMINISTRACIÓN	34
4.1 Control de Accesos y Seguridad de Roles (PostgreSQL Privilege Management).....	34
4.2 Auditoría Persistente de Modificación de Datos	35
4.3 Estrategia de Respaldo y Recuperación (Disaster Recovery Plan)	35

RESUMEN EJECUTIVO

El rol de los datos en el sistema "FINANZAS-MED" constituye el pilar fundamental para garantizar la estabilidad operativa, la rentabilidad financiera y la auditoría de seguridad del negocio. Al tratarse de un ERP Clínico y Financiero que procesa cobros inmediatos (copagos), deudas diferidas (coaseguros con aseguradoras), inventarios críticos de medicamentos (Kardex permanente) y el arqueo de fondos de bóveda, la base de datos no puede limitarse a actuar como un simple almacén pasivo de persistencia.

Por el contrario, la base de datos está diseñada como una **Plataforma de Datos Activa y Transaccional**. Esto significa que:

1. **Garantiza la Integridad Referencial Estricta:** La base de datos aplica restricciones físicas (FOREIGN KEY, CHECK constraints, NOT NULL) para evitar estados inconsistentes (como un movimiento de caja chica sin un método de pago válido o una venta sin sucursal asignada).
2. **Mitiga Condiciones de Carrera Concurrentes:** Mediante el bloqueo de filas en transacciones críticas (SELECT ... FOR UPDATE), se impide que solicitudes concurrentes de múltiples cajeros generen descuadres contables o duplicidades de stock.
3. **Ejecuta Lógica de Negocio Crítica a Nivel de Motor (Triggers):** La lógica de actualización de saldos contables de caja y movimientos de inventario se programa en el motor mediante funciones **PL/pgSQL**. Esto independiza la consistencia de los datos ante cualquier fallo o caída del servidor de aplicaciones Spring Boot.
4. **Optimiza el Rendimiento y la Auditoría:** El sistema incorpora índices B-Tree específicos para búsquedas analíticas frecuentes y un flujo automatizado de auditoría inmutable que registra cada escritura en un repositorio seguro (SystemAuditLog).

SECCIÓN 1: MODELO DE DATOS Y MAPEO DE CAPTURAS

Esta sección presenta los diagramas relacionales físicos extraídos directamente del motor de la base de datos PostgreSQL, clasificados por sus módulos de negocio.

1.1 Módulo de Seguridad, Accesos y Menú Dinámico

Este módulo gestiona la autenticación de usuarios, la segregación de roles y la carga en tiempo real del menú dinámico de la barra lateral en el frontend de Angular.

A. Relación de Roles de Usuario

El diagrama muestra la tabla asociativa `user_roles` que vincula de manera N-a-N a los usuarios del sistema (`datos_personales`) con sus perfiles de permisos autorizados (`roles`).

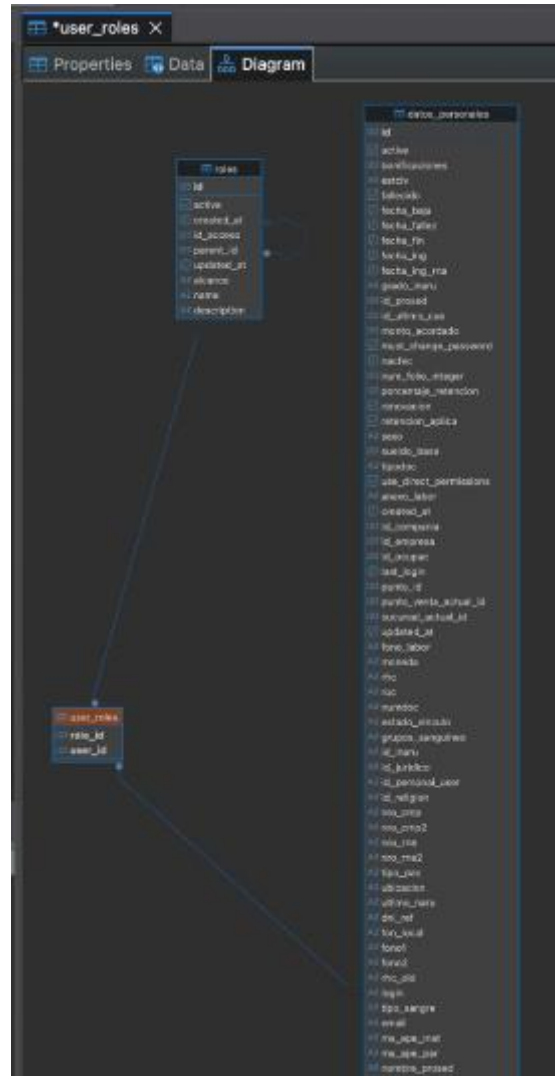
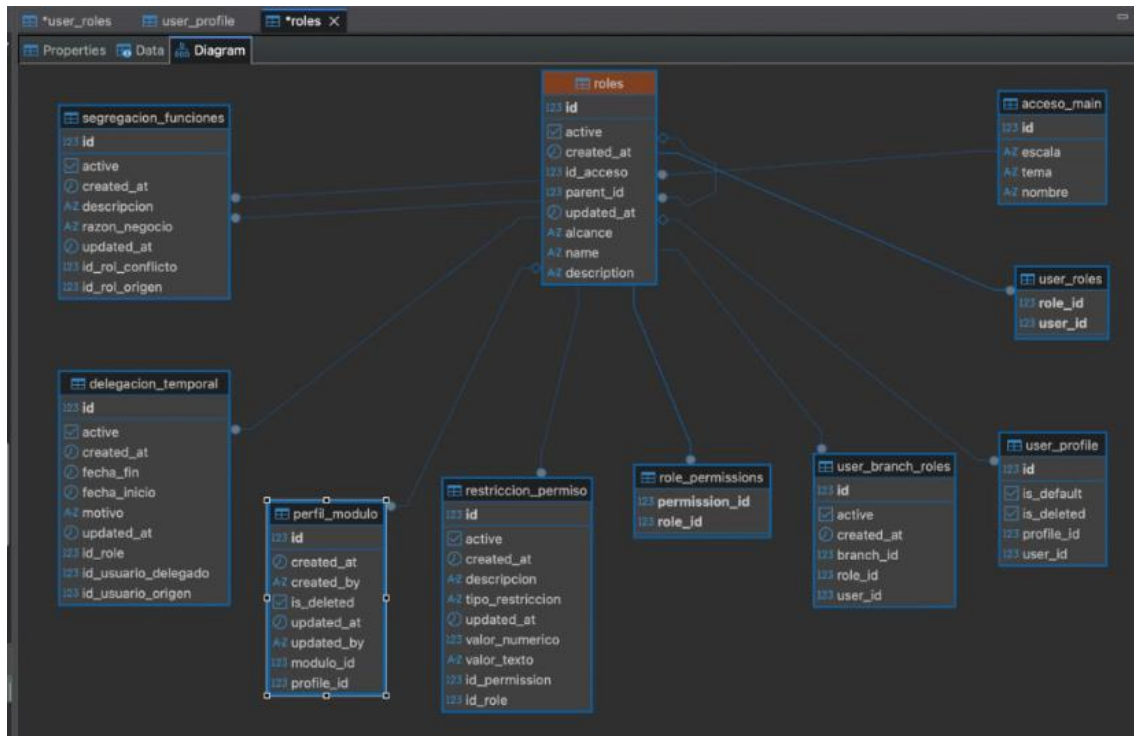


Diagrama de Roles de Usuario

B. Asignación Multi-sucursal y Restricciones de Segregación

Mapeo de privilegios de usuario por sede geográfica mediante la tabla `user_branch_roles` y las restricciones de seguridad internas.



Seguridad y Asignación Multi-sucursal

C. Estructura de Menú Dinámico

Diagrama de navegación lateral consumido por la SPA de Angular. Las tablas menu y modulo se vinculan con los roles para cargar de forma adaptativa las rutas del sistema.

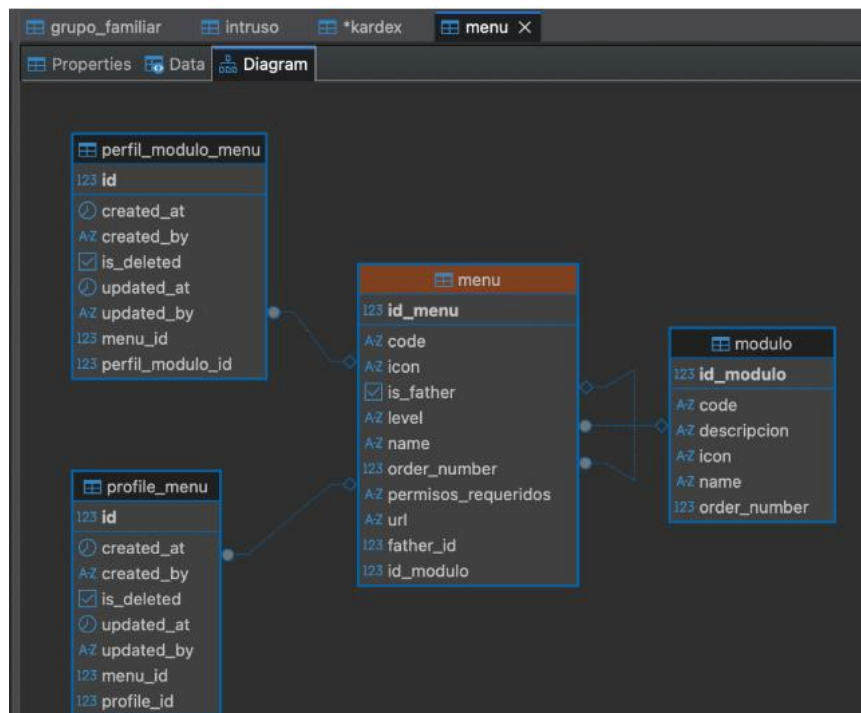
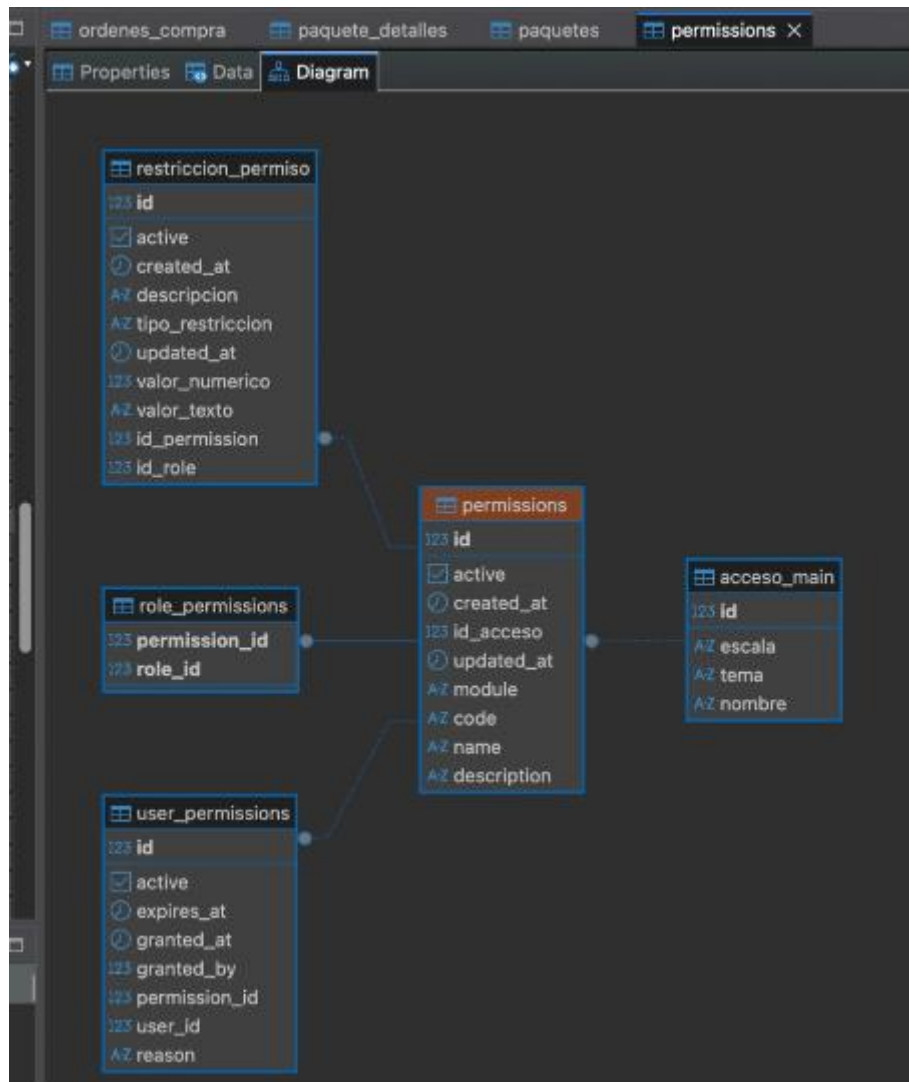


Diagrama del Menú Dinámico

D. Permisos Granulares y Excepciones

Diagrama que expone la tabla permissions y su vinculación con los permisos de usuario individuales (user_permissions) con vigencia y expiración programada.



Permisos Granulares y Excepciones

1.2 Módulo de Catálogo General, Productos y Kardex Logístico

Maneja el maestro unificado de servicios médicos y productos farmacéuticos, controlando el stock y los movimientos de inventario de medicamentos por lotes.

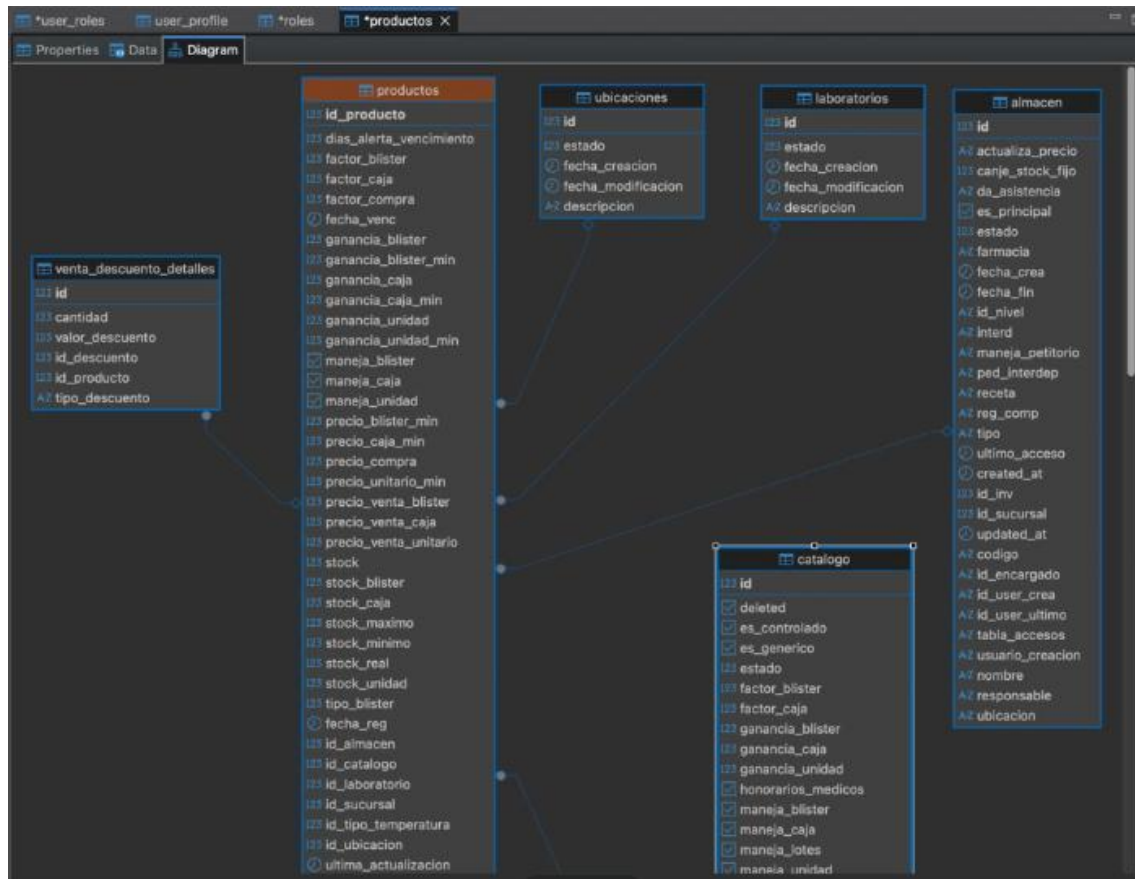
A. Catálogo General (Sección Superior)

Estructura de la tabla catalogo que actúa como base heredada para productos y servicios asistenciales de la clínica.

Catálogo General - Sección Inferior

C. Ficha de Productos e Inventarios

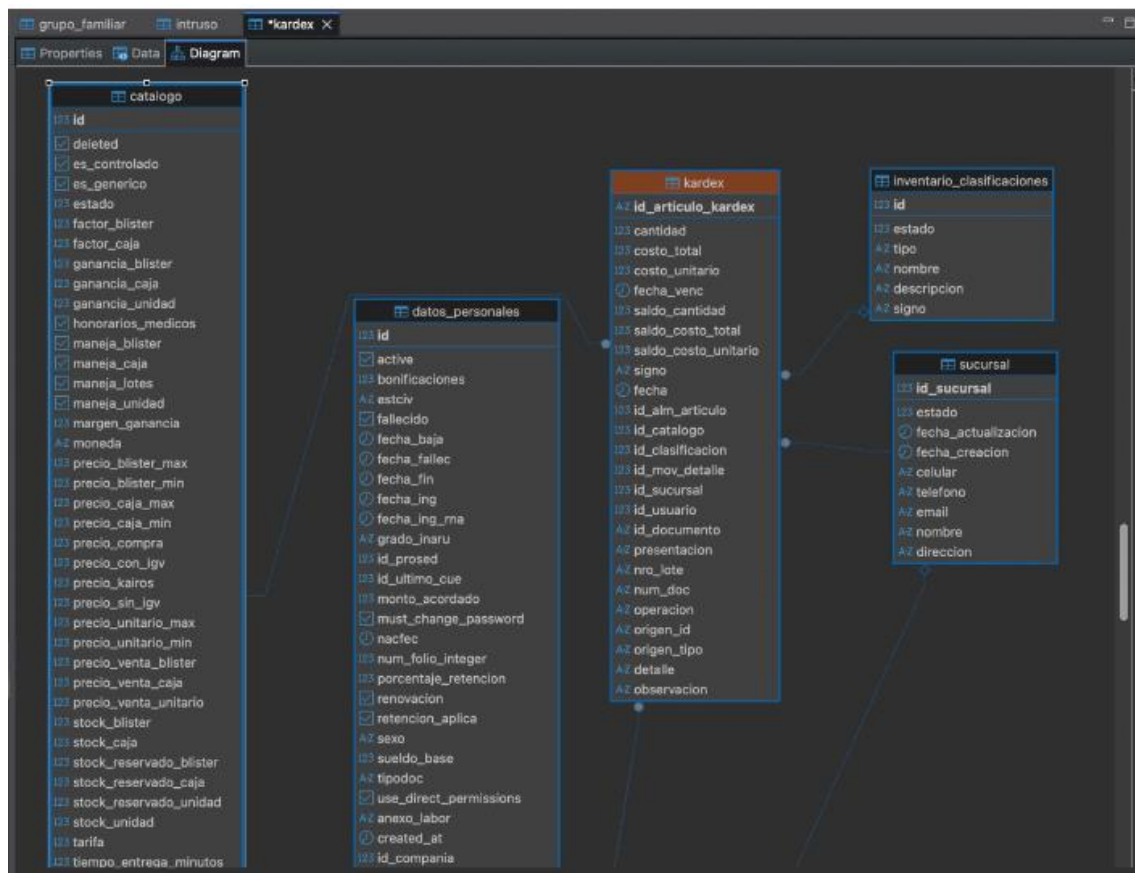
Relación 1-a-1 de la tabla productos con catálogo, incorporando variables de control de stock mínimo, stock máximo, fecha de vencimiento y laboratorios.



Ficha de Productos e Inventario

D. Control de Kardex Permanente

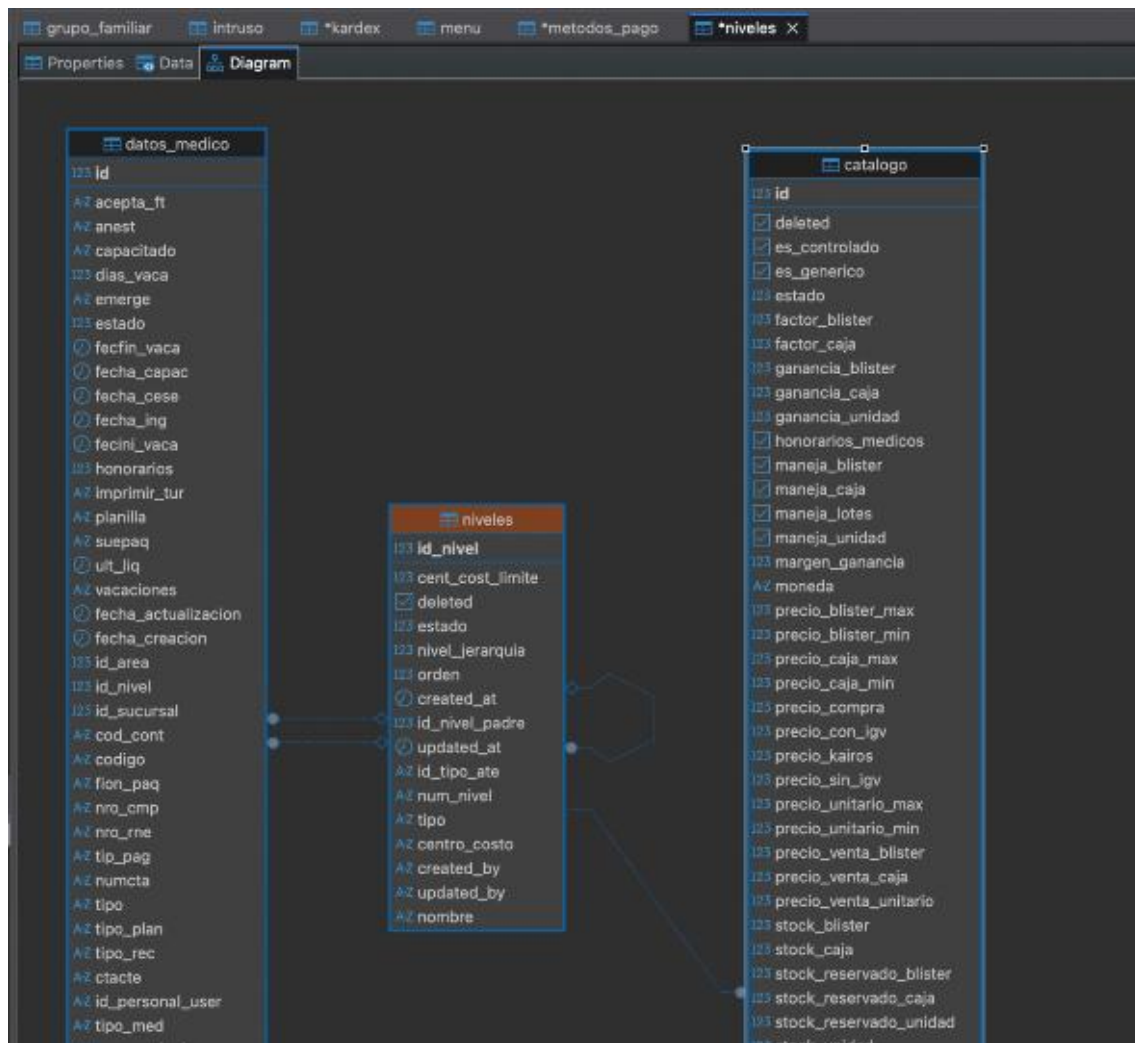
Diagrama de la tabla transaccional de Kardex que registra cada entrada y salida física con su respectiva sucursal, cantidad, costo e identificador de usuario.



Kardex Permanente

E. Niveles y Tarifarios Médicos

Asociación del catálogo de servicios con los médicos tratantes para la asignación dinámica de tarifas y honorarios profesionales según su nivel de especialidad.



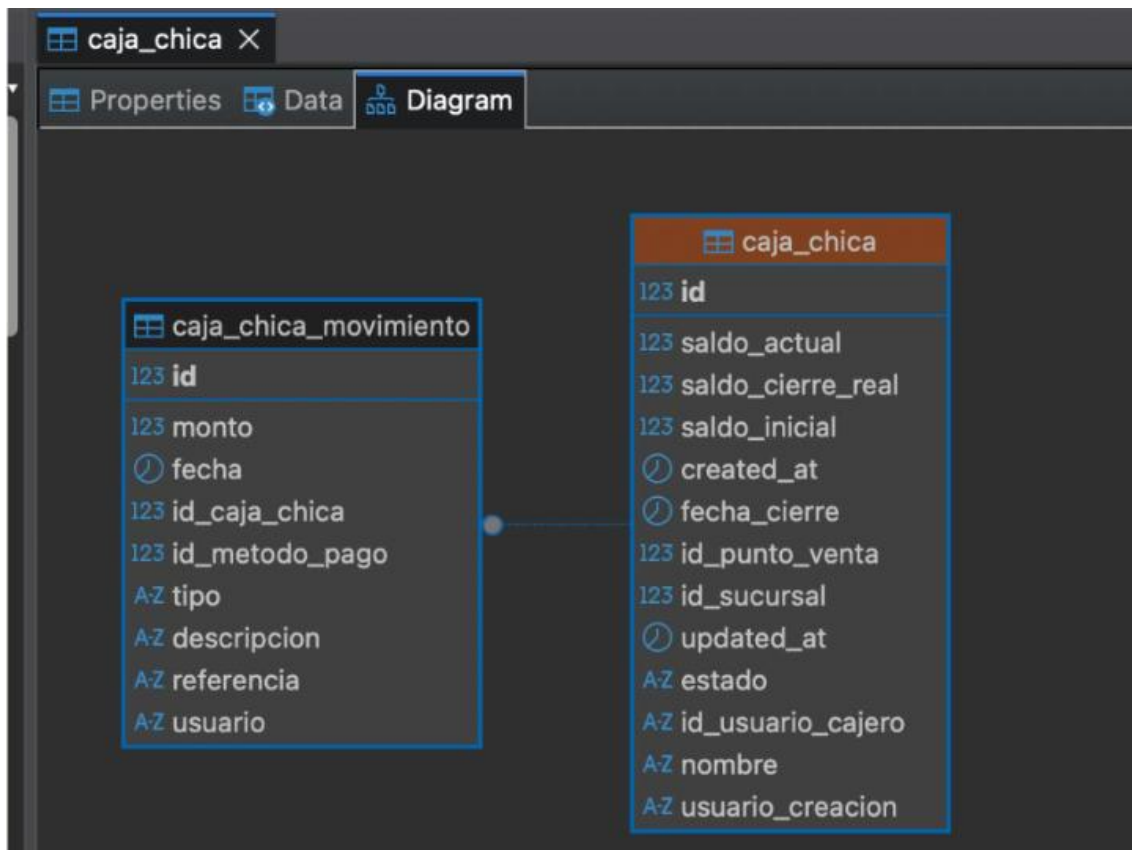
Niveles y Tarifas

1.3 Módulo de Tesorería, Cajas Chicas y Caja General

Controla la recaudación de ventanilla diaria en los puntos de venta y la posterior transferencia consolidada a la bóveda central de la sucursal.

A. Caja Chica Cabecera

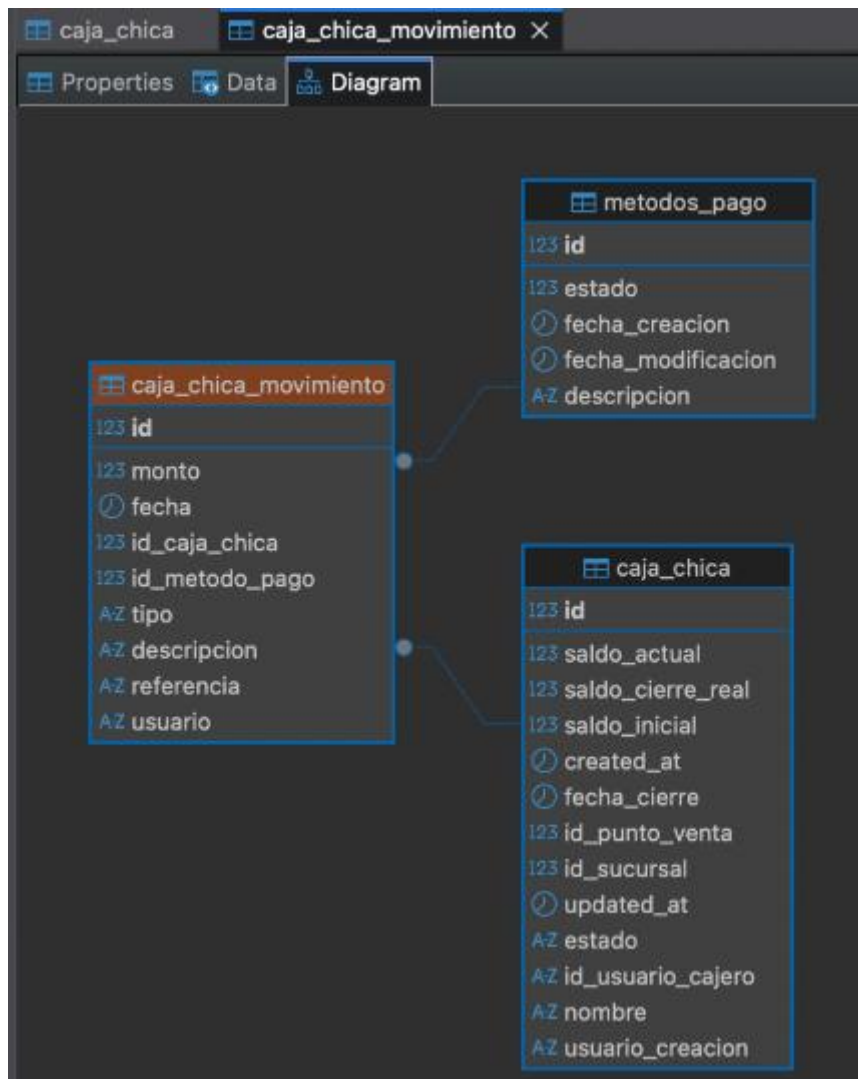
Estructura de control diario de saldos contables y de estado de apertura/cierre de la caja de ventanilla.



Cabecera de Caja Chica

B. Movimientos de Caja Chica

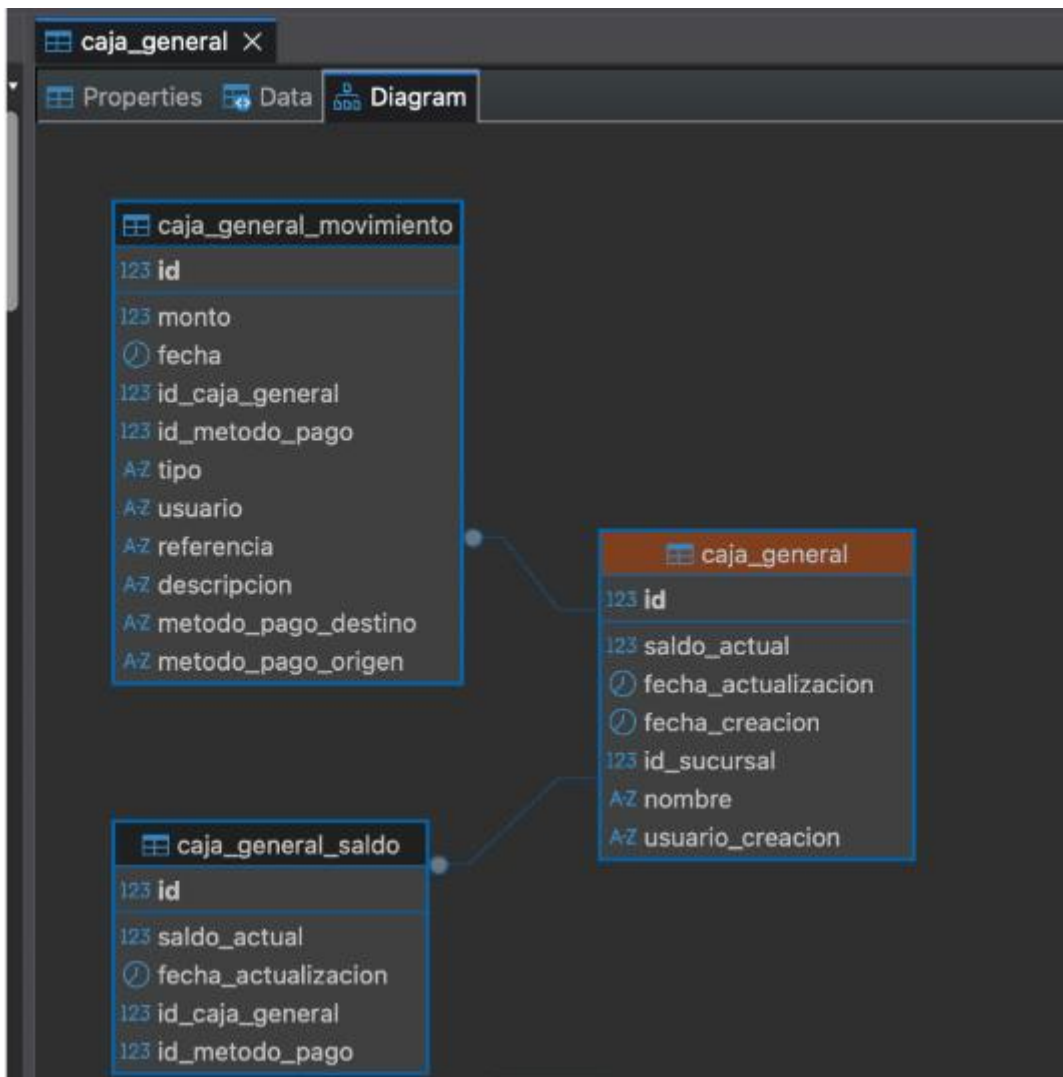
Registro analítico de ingresos y egresos de caja chica vinculados de manera obligatoria a un método de pago.



Movimientos de Caja Chica

C. Caja General (Bóveda)

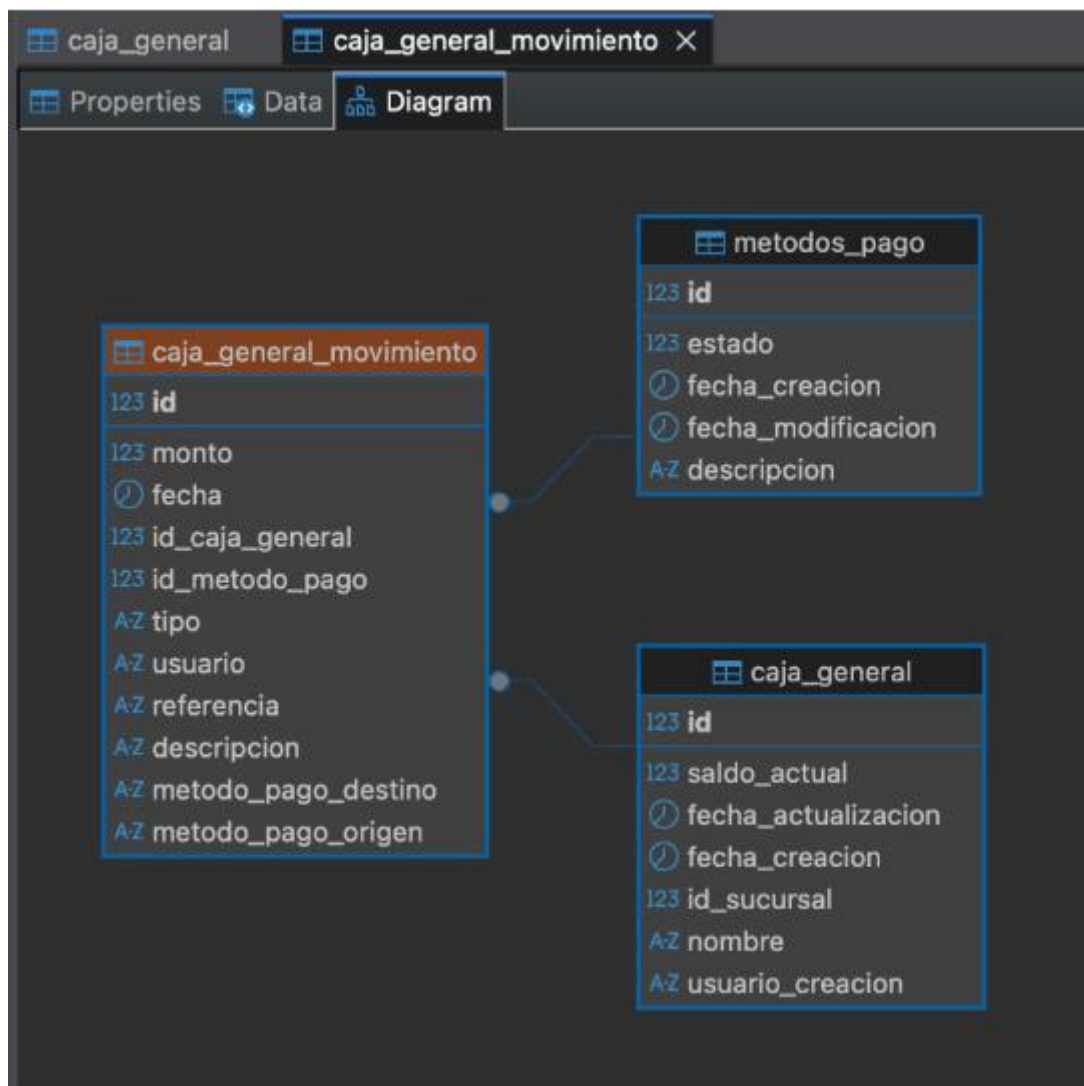
Diagrama de la bóveda centralizadora de sucursal que recibe los fondos acumulados al momento del arqueo de cajas chicas.



Caja General - Bóveda

D. Movimientos de Caja General

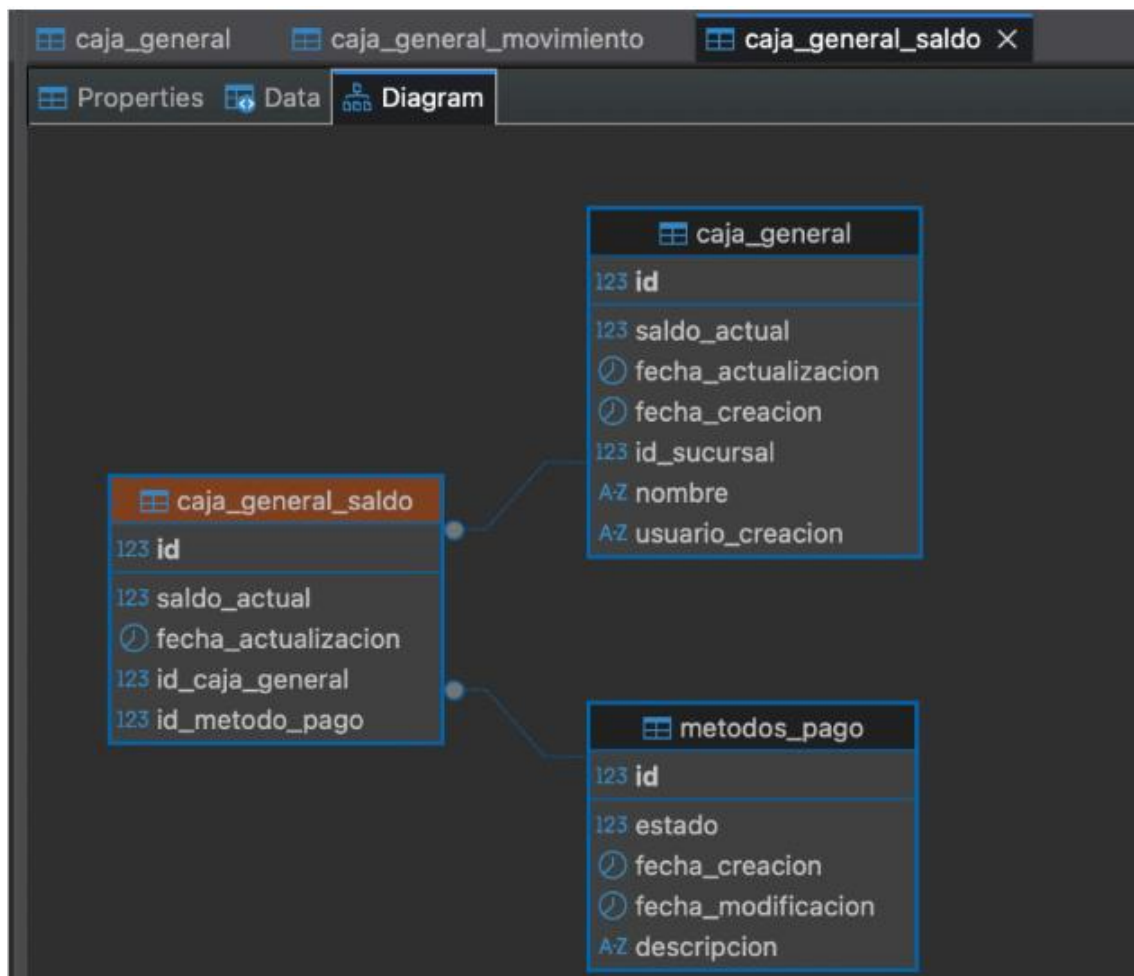
Historial de entradas (transferencias por cierres) y egresos mayores desde la bóveda hacia cuentas bancarias corporativas.



Movimientos de Caja General

E. Saldos Consolidados por Método de Pago

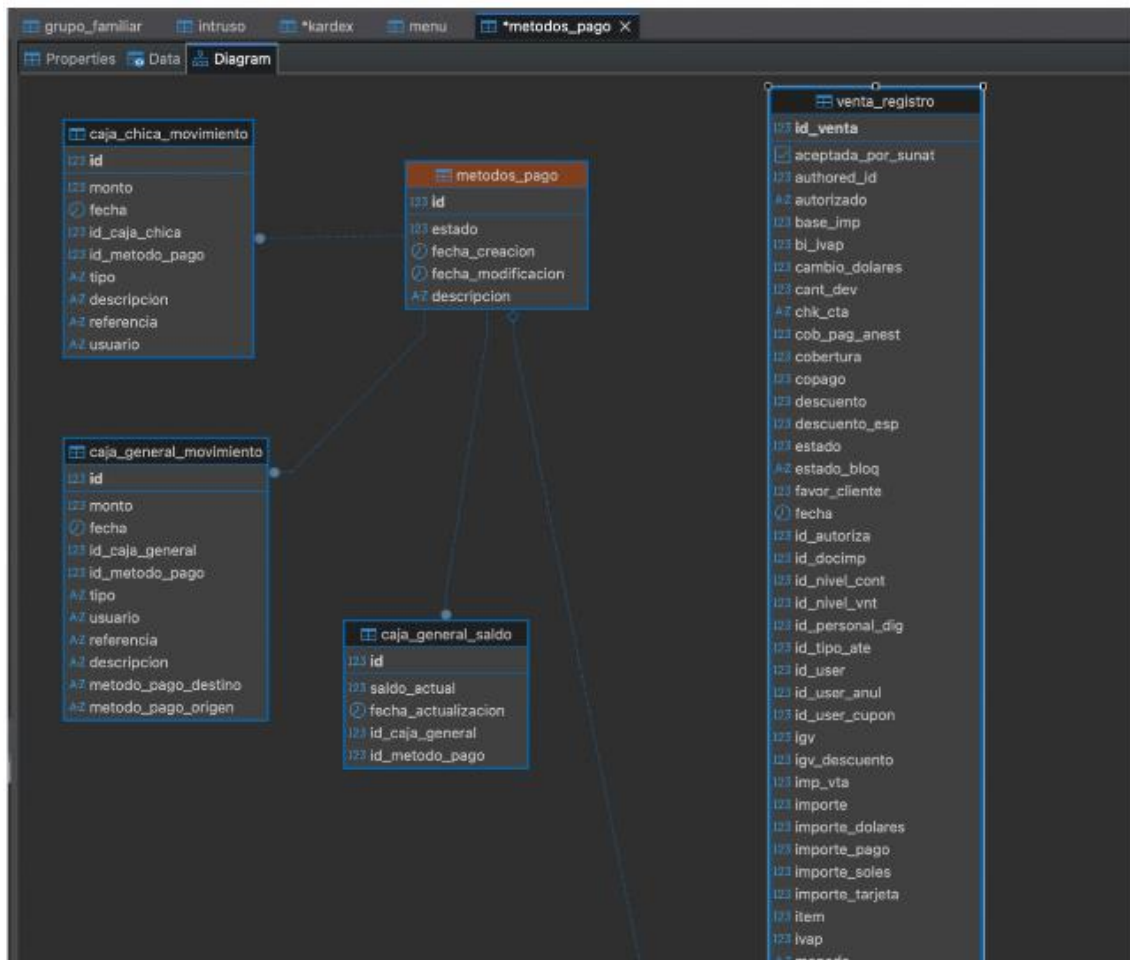
Tabla relacional `caja_general_saldo` que acumula los fondos de la bóveda discriminados por tipo de cobro (Efectivo, Tarjetas, Yape).



Saldos por Método de Pago

F. Catálogo Central de Métodos de Pago

Vínculo centralizador que estandariza las modalidades de pago aceptadas en todas las transacciones de ventas y egresos.



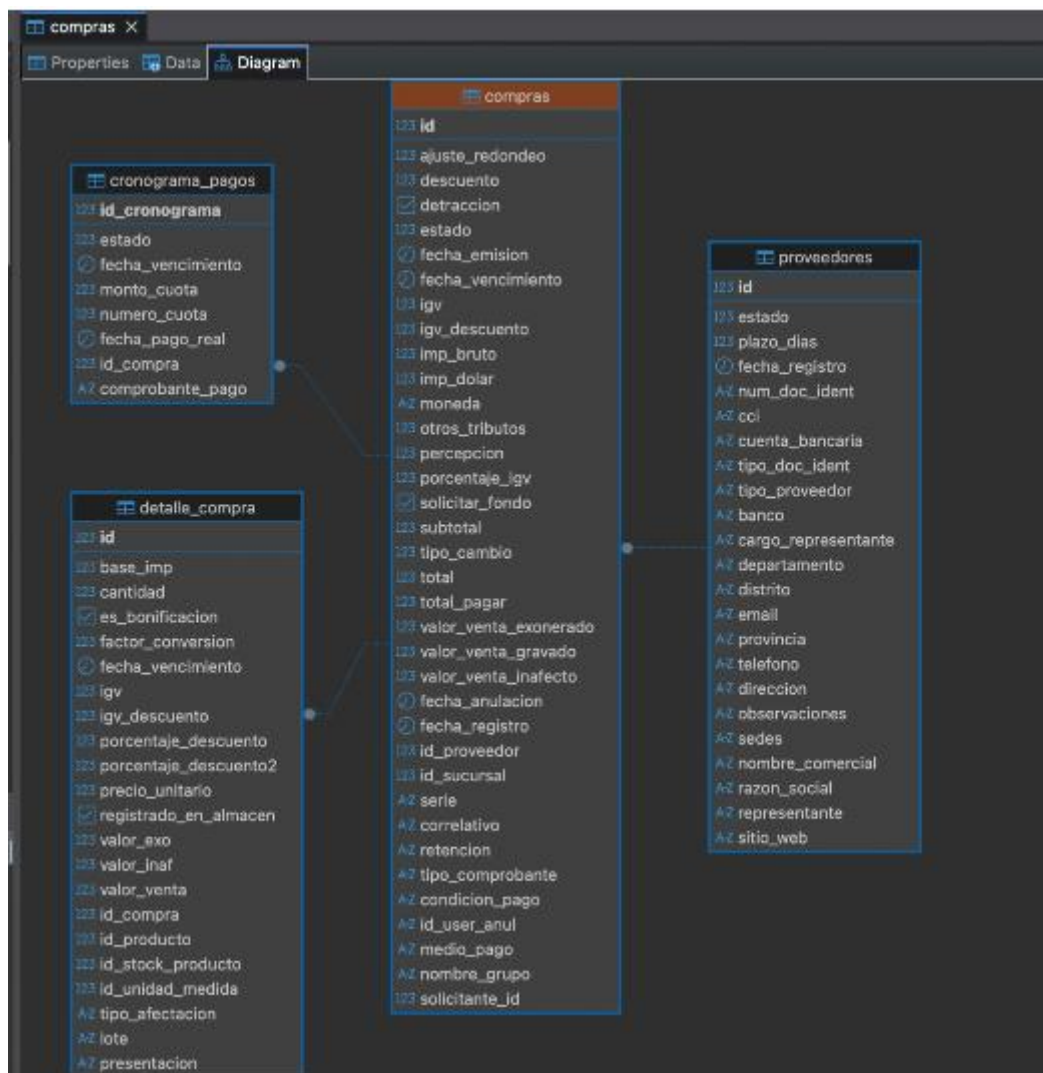
Catálogo de Métodos de Pago

1.4 Módulo de Logística y Compras a Proveedores

Administra el proceso de reabastecimiento, desde la cotización y aprobación de órdenes hasta el registro de las facturas de proveedores a crédito.

A. Registro de Compras (Facturas)

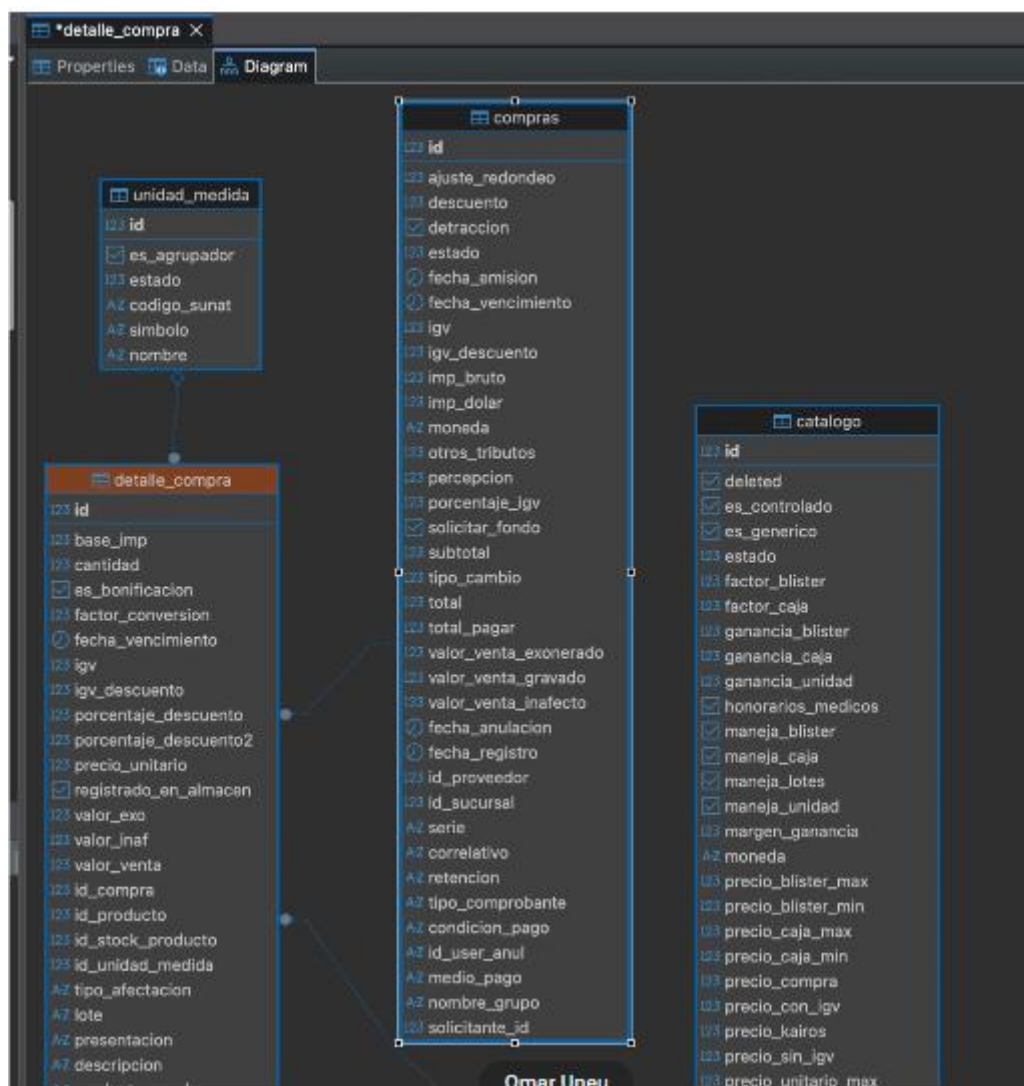
Tabla cabecera compras que vincula la factura tributaria física con el proveedor y los impuestos aplicados.



Facturas de Compras

B. Detalle Físico de Compras

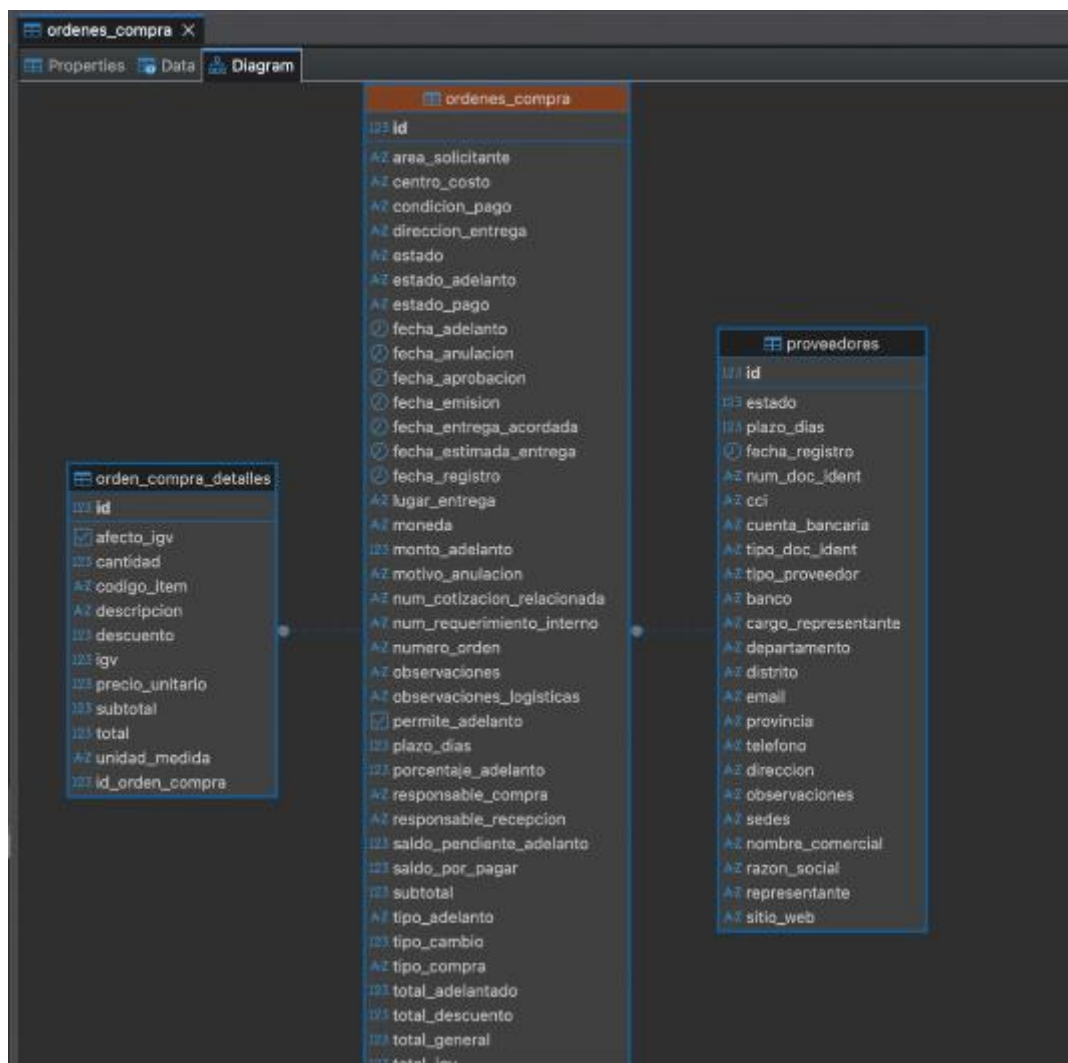
Línea de detalle detalle_compra que asocia los productos del catálogo con sus unidades de medida, costos y cantidades ingresadas al almacén.



Detalle de Compras

C. Emisión de Órdenes de Compra

Diagrama del flujo logístico inicial que detalla el requerimiento formal a proveedores pendiente de aprobación de gerencia.



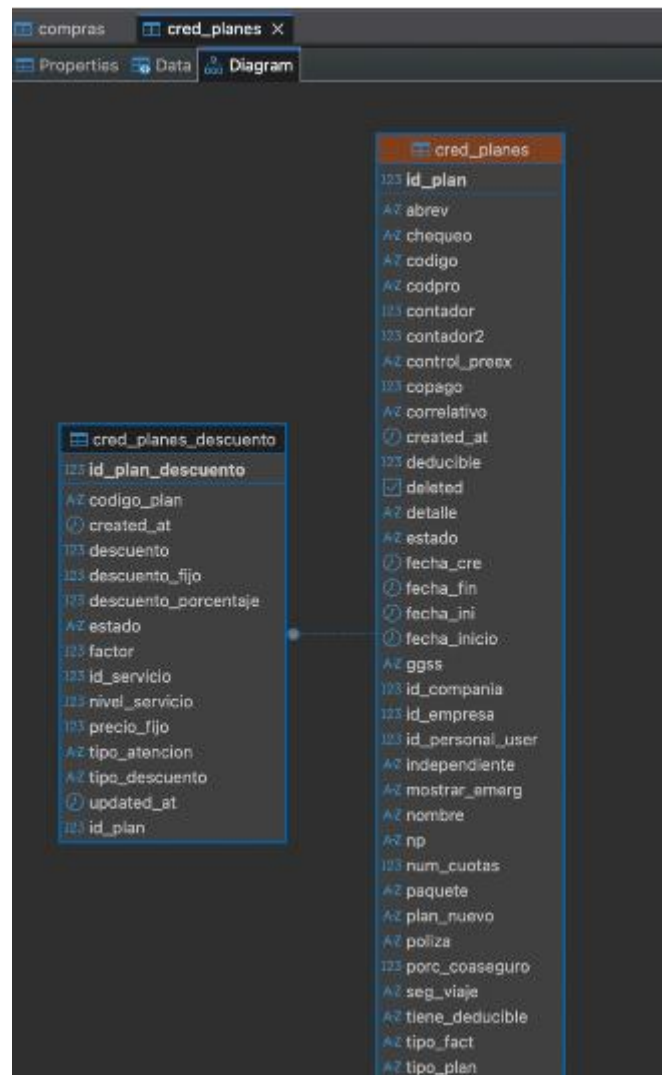
Órdenes de Compra

1.5 Módulo de Cuentas Corrientes, Convenios y Paquetes

Módulo que procesa la liquidación con las compañías aseguradoras, regulando las coberturas de coaseguro y consumos acumulados de pacientes corporativos.

A. Cuentas Corrientes de Convenio

Diagrama de la tabla cred_ctacte que registra los límites y el consumo acumulado de pólizas de seguros.



Cuentas Corrientes de Convenio

B. Cronograma de Pagos y Amortización

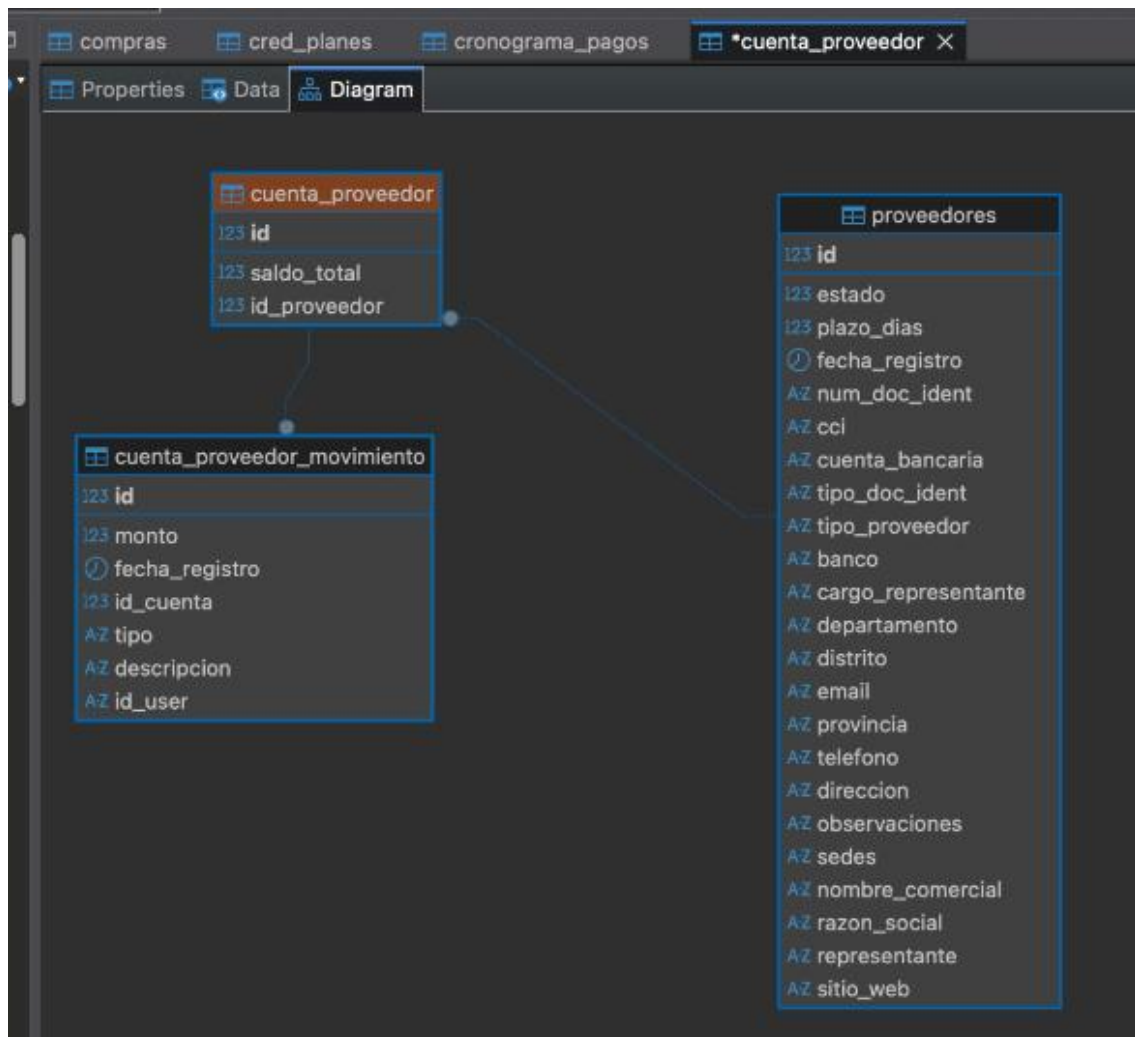
Programación de fechas de vencimiento de obligaciones financieras generadas al crédito.



Cronograma de Pagos

C. Cuenta Proveedor Acumulada

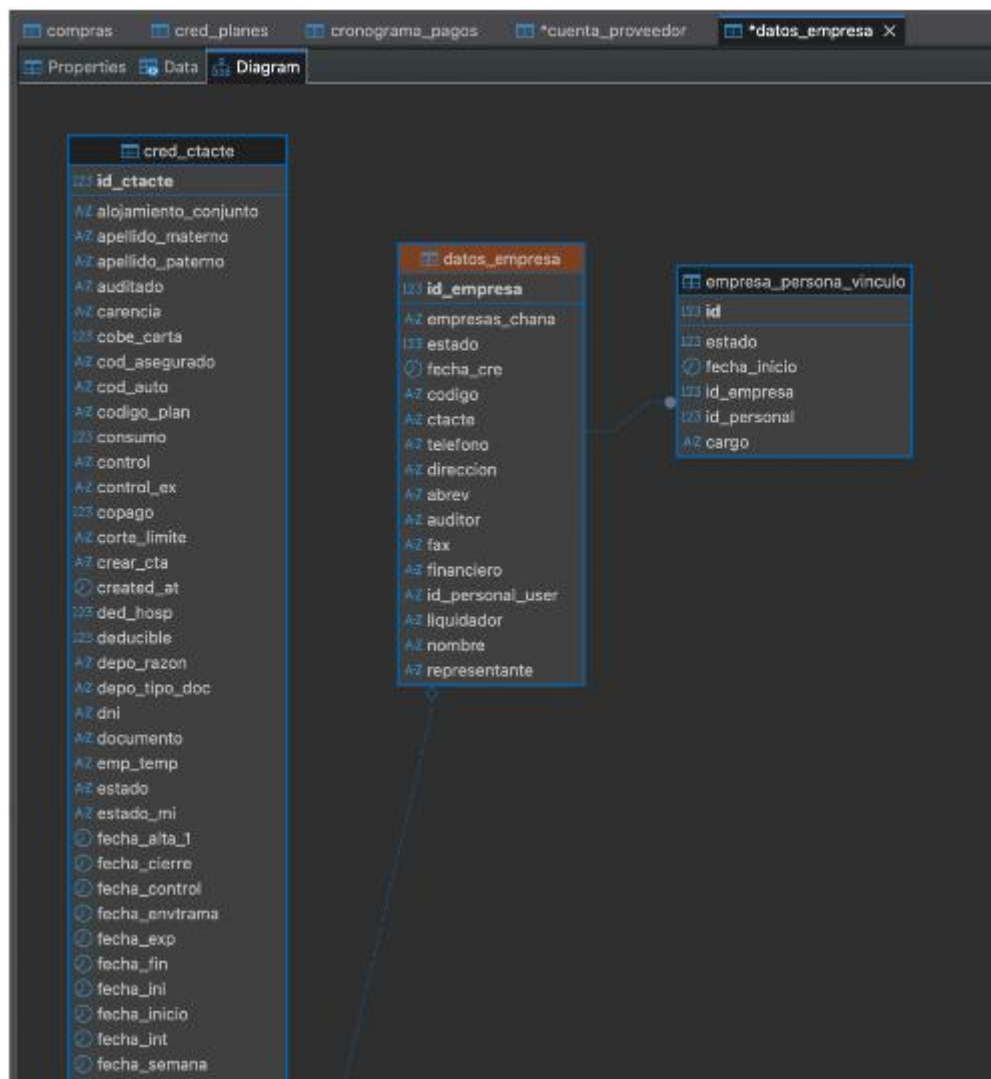
Tabla de consolidación de deuda total por pagar a cada droguería o laboratorio proveedor.



Cuenta Proveedor Acumulada

D. Datos de Empresas Conveniadas

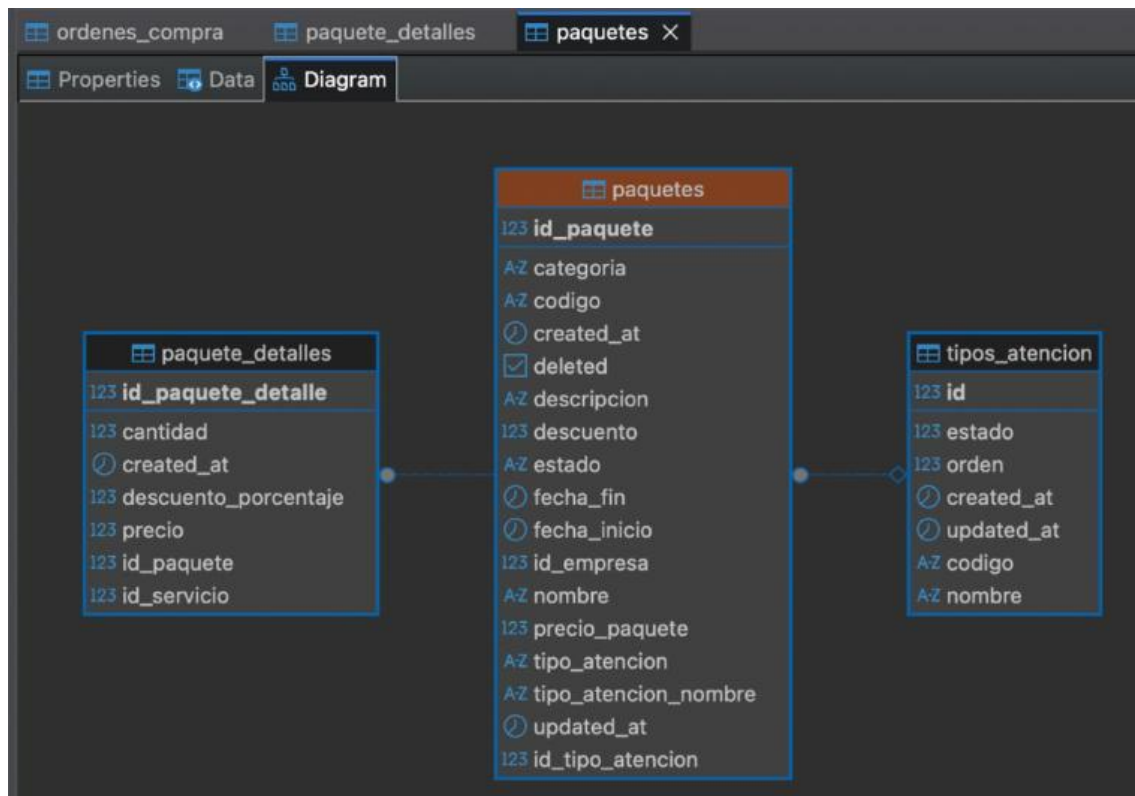
Relación de las personas jurídicas (datos_empresa) vinculadas a los pacientes y sus planes de cobertura.



Datos de Empresas

E. Oferta de Paquetes Promocionales

Estructura de la tabla paquetes que define los servicios asistenciales paquetizados con tarifas cerradas y vigencia temporal.



Paquetes Promocionales

1.6 Módulo de Facturación y Consolidación de Sucursal

Centraliza la emisión final de comprobantes electrónicos de venta ante SUNAT y los desglosa en copagos en efectivo y cuentas por cobrar a seguros.

A. Estructura de Sucursales (Eje Centralizador)

Eje geográfico multi-sucursal que vincula de forma directa la procedencia de todas las ventas, inventarios y cajas de la clínica.



Detalle de Venta Asistencial

1.7 Justificación de la Normalización de Base de Datos

Para evitar redundancias e inconsistencias de datos, el modelo de datos de "FINANZAS-MED" se ha normalizado estrictamente hasta la **Tercera Forma Normal (3FN)**.

Caso Práctico de Normalización: Tabla catalogo y productos

- **Primera Forma Normal (1FN):** Se garantiza que todas las columnas contengan únicamente valores atómicos y que no existan grupos repetidos. Por ejemplo, los precios de venta unitario, blister y caja se definen en columnas individuales y con tipos de datos atómicos (DECIMAL).
- **Segunda Forma Normal (2FN):** Para cumplir con la 2FN, el esquema debe estar en 1FN y todas las columnas que no forman parte de la clave primaria

deben depender por completo de la clave primaria completa. Se aplicó la 2FN dividiendo la tabla en:

- catalogo: Atributos comunes a todos los ítems (Nombre, Tipo, Códigos Tributarios, Precios de Venta).
- productos: Atributos específicos de medicamentos físicos (Stock, Stock Mínimo, Fecha de Vencimiento, Factor Caja).
- **Tercera Forma Normal (3FN):** El esquema debe estar en 2FN y no debe existir ninguna dependencia transitiva de las columnas no primarias respecto a la clave primaria. Se normalizó extrayendo la información del laboratorio a una tabla independiente laboratorios con clave primaria id, y en la tabla productos se almacena únicamente el campo id_laboratorio como FK.

SECCIÓN 2: IMPLEMENTACIÓN DE BASE DE DATOS (SCRIPTS DDL & DML)

Esta sección expone los scripts SQL necesarios para implementar la estructura relacional y cargar la información semilla para el arranque del ERP.

2.1 Scripts DDL (PostgreSQL)

```
-- Habilitar extensión para auditorías temporales si se requiere
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
```

```
-- 1. Tabla de Sucursales
CREATE TABLE sucursal (
    id_sucursal BIGSERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    direccion VARCHAR(200),
    estado VARCHAR(1) DEFAULT 'A' CHECK (estado IN ('A', 'I')),
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 2. Tabla de Datos Personales (Usuarios)
CREATE TABLE datos_personales (
    id BIGSERIAL PRIMARY KEY,
    login VARCHAR(50) NOT NULL UNIQUE,
    passwd VARCHAR(255) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    nombre VARCHAR(100) NOT NULL,
    apepat VARCHAR(100) NOT NULL,
    apemat VARCHAR(100) NOT NULL,
    sueldo_base DECIMAL(12, 4) DEFAULT 0.0000,
    active BOOLEAN DEFAULT TRUE,
    sucursal_actual_id INTEGER,
    last_login TIMESTAMP,
```

```

        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

-- 3. Tabla de Catálogo General
CREATE TABLE catalogo (
    id BIGSERIAL PRIMARY KEY,
    nombre VARCHAR(200) NOT NULL,
    tipo_catalogo VARCHAR(50) NOT NULL CHECK (tipo_catalogo IN
('PRODUCTO', 'SERVICIO')),
    precio_venta_unitario DECIMAL(12, 4) NOT NULL DEFAULT 0.0000,
    precio_compra DECIMAL(12, 4) NOT NULL DEFAULT 0.0000,
    es_controlado BOOLEAN DEFAULT FALSE,
    cod_digemid VARCHAR(50),
    codigo_susalud VARCHAR(50)
);

-- 4. Tabla de Productos Farmacéuticos (Herencia 1-a-1 de catálogo)
CREATE TABLE productos (
    id_producto BIGINT PRIMARY KEY REFERENCES catalogo(id) ON DELETE
CASCADE,
    stock DECIMAL(12, 4) NOT NULL DEFAULT 0.0000,
    stock_minimo DECIMAL(12, 4) NOT NULL DEFAULT 0.0000,
    stock_maximo DECIMAL(12, 4) NOT NULL DEFAULT 0.0000,
    fecha_venc DATE,
    id_laboratorio BIGINT,
    factor_caja DECIMAL(10, 2) NOT NULL DEFAULT 1.00
);

-- 5. Tabla de Métodos de Pago
CREATE TABLE metodos_pago (
    id BIGSERIAL PRIMARY KEY,
    descripcion VARCHAR(255) NOT NULL UNIQUE,
    estado INTEGER DEFAULT 1 CHECK (estado IN (0, 1)),
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 6. Tabla de Caja Chica (Cabecera)
CREATE TABLE caja_chica (
    id BIGSERIAL PRIMARY KEY,
    nombre VARCHAR(255) NOT NULL,
    saldo_inicial DECIMAL(15, 5) NOT NULL DEFAULT 0.00000,
    saldo_actual DECIMAL(15, 5) NOT NULL DEFAULT 0.00000,
    saldo_cierre_real DECIMAL(15, 5),
    estado VARCHAR(50) DEFAULT 'ABIERTA' CHECK (estado IN ('ABIERTA',
'CERRADA')),
    id_usuario_cajero VARCHAR(255) NOT NULL,
    fecha_cierre TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 7. Tabla de Movimientos de Caja Chica
CREATE TABLE caja_chica_movimiento (
    id BIGSERIAL PRIMARY KEY,
    id_caja_chica BIGINT NOT NULL REFERENCES caja_chica(id) ON DELETE
RESTRICT,
    tipo VARCHAR(50) NOT NULL CHECK (tipo IN ('INGRESO', 'EGRESO')),

```

```

        monto DECIMAL(15, 5) NOT NULL CHECK (monto > 0),
        id_metodo_pago BIGINT NOT NULL REFERENCES metodos_pago(id) ON
DELETE RESTRICT,
        descripcion TEXT,
        referencia VARCHAR(255),
        usuario VARCHAR(255) NOT NULL,
        fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

```

-- 8. Tabla de Registro de Auditoría

```

CREATE TABLE system_audit_log (
    id BIGSERIAL PRIMARY KEY,
    usuario VARCHAR(100) NOT NULL,
    ip_address VARCHAR(45) NOT NULL,
    tabla_afectada VARCHAR(100) NOT NULL,
    operacion VARCHAR(20) NOT NULL CHECK (operacion IN ('INSERT',
'UPDATE', 'DELETE')),
    valor_anterior TEXT,
    valor_nuevo TEXT,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

2.2 Scripts DML (Carga Semilla)

-- Cargar sucursales de prueba

```

INSERT INTO sucursal (nombre, direccion) VALUES
('Sede Principal - Lima Centro', 'Av. Wilson 1230, Lima'),
('Sede San Isidro', 'Av. Javier Prado Este 450, San Isidro'),
('Sede Miraflores', 'Av. Larco 780, Miraflores');

```

-- Cargar métodos de pago admitidos

```

INSERT INTO metodos_pago (descripcion) VALUES
('EFFECTIVO'), ('YAPE'), ('PLIN'), ('TARJETA_CREDITO'),
('TARJETA_DEBITO'), ('TRANSFERENCIA_BANCARIA');

```

-- Cargar catálogo semilla (Servicios y Medicamentos)

```

INSERT INTO catalogo (nombre, tipo_catalogo, precio_venta_unitario,
precio_compra, es_controlado, cod_digemid) VALUES
('Consulta Medica General', 'SERVICIO', 50.0000, 0.0000, FALSE, NULL),
('Examen de Sangre - Hemograma Completo', 'SERVICIO', 45.0000,
10.0000, FALSE, NULL),
('Amoxicilina 500mg (Caja x 100)', 'PRODUCTO', 35.0000, 15.0000,
FALSE, 'F12409'),
('Alprazolam 0.5mg (Caja x 30)', 'PRODUCTO', 45.0000, 20.0000, TRUE,
'F15671');

```

-- Asociar información de inventario físico a los productos cargados

```

INSERT INTO productos (id_producto, stock, stock_minimo, stock_maximo,
fecha_venc, factor_caja) VALUES
(3, 150.0000, 10.0000, 500.0000, '2027-12-31', 1.00),
(4, 80.0000, 5.0000, 200.0000, '2026-11-30', 1.00);

```

-- Cargar usuario administrador base (Contraseña cifrada con Bcrypt para 'admin123')

```

INSERT INTO datos_personales (login, passwd, email, nombre, apepat,
apemat, sueldo_base, sucursal_actual_id) VALUES

```

```
('admin',  
'$2a$12$Zp40o9n6N/8hWl9jE.e8eOsznlK4/Z6l8V7M3Z2rL3.g77H05F8uG',  
'sistemas@clinica.pe', 'Omar', 'Condor', 'Upeu', 3500.0000, 1);
```

SECCIÓN 3: CONSULTAS Y PROGRAMACIÓN (TRIGGERS & PROCEDURES)

La programación en el motor de base de datos asegura que la integridad de los saldos y de los inventarios se controle de manera directa por PostgreSQL.

3.1 Triggers Transaccionales en PL/pgSQL

Trigger 1: Actualización del Saldo de Caja Chica

Este trigger actualiza de forma automática el saldo acumulado en la tabla `caja_chica` cuando se inserta un nuevo movimiento en la tabla `caja_chica_movimiento`.

```
CREATE OR REPLACE FUNCTION fn_actualizar_saldo_caja_chica()  
RETURNS TRIGGER AS $$  
BEGIN  
    -- Bloquear la fila de la cabecera de caja chica para evitar  
    condiciones de carrera (lecturas sucias)  
    PERFORM id FROM caja_chica WHERE id = NEW.id_caja_chica FOR  
    UPDATE;  
  
    -- Sumar o restar al saldo contable según el tipo de movimiento  
    IF NEW.tipo = 'INGRESO' THEN  
        UPDATE caja_chica  
        SET saldo_actual = saldo_actual + NEW.monto,  
            updated_at = CURRENT_TIMESTAMP  
        WHERE id = NEW.id_caja_chica;  
    ELSIF NEW.tipo = 'EGRESO' THEN  
        UPDATE caja_chica  
        SET saldo_actual = saldo_actual - NEW.monto,  
            updated_at = CURRENT_TIMESTAMP  
        WHERE id = NEW.id_caja_chica;  
    END IF;  
  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE OR REPLACE TRIGGER tg_actualizar_saldo_caja_chica  
AFTER INSERT ON caja_chica_movimiento  
FOR EACH ROW  
EXECUTE FUNCTION fn_actualizar_saldo_caja_chica();
```

Trigger 2: Registro de Trazabilidad e Inmutabilidad en Kardex

Este trigger sincroniza el stock en la tabla `productos` cuando se registra un movimiento físico en el kardex.

```

CREATE OR REPLACE FUNCTION fn_sincronizar_stock_kardex()
RETURNS TRIGGER AS $$
BEGIN
    -- Validar si el artículo del catálogo corresponde a un producto
    físico
    IF NOT EXISTS (SELECT 1 FROM productos WHERE id_producto =
NEW.id_catálogo) THEN
        RETURN NEW;
    END IF;

    -- Bloquear la fila del producto para concurrencia
    PERFORM id_producto FROM productos WHERE id_producto =
NEW.id_catálogo FOR UPDATE;

    -- Sumar o restar stock según el signo del movimiento del Kardex
    IF NEW.signo = '+' THEN
        UPDATE productos
        SET stock = stock + NEW.cantidad
        WHERE id_producto = NEW.id_catálogo;
    ELSIF NEW.signo = '-' THEN
        UPDATE productos
        SET stock = stock - NEW.cantidad
        WHERE id_producto = NEW.id_catálogo;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER tg_sincronizar_stock_kardex
AFTER INSERT ON kardex
FOR EACH ROW
EXECUTE FUNCTION fn_sincronizar_stock_kardex();

```

3.2 Funciones y Procedimientos Almacenados (PL/pgSQL)

Procedimiento para Registrar Compra a Proveedor y Generar Cuotas de Pago

Este procedimiento automatiza la creación de una compra, actualiza la cuenta por pagar del proveedor e inserta las cuotas programadas al crédito.

```

CREATE OR REPLACE PROCEDURE pr_registrar_compra_credito(
    p_id_proveedor BIGINT,
    p_total DECIMAL(12, 4),
    p_num_cuotas INT,
    p_fecha_emision DATE,
    p_plazo_dias INT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_id_compra BIGINT;
    v_monto_cuota DECIMAL(12, 4);
    i INT;
BEGIN
    v_monto_cuota := p_total / p_num_cuotas;

```

```

-- 1. Insertar cabecera de compra
INSERT INTO compras (id_proveedor, total, condicion_pago,
fecha_emision, fecha_vencimiento)
VALUES (p_id_proveedor, p_total, 'CREDITO', p_fecha_emision,
p_fecha_emision + p_plazo_dias)
RETURNING id INTO v_id_compra;

-- 2. Actualizar o insertar saldo en la cuenta corriente del
proveedor
INSERT INTO cuenta_proveedor (id_proveedor, saldo_total)
VALUES (p_id_proveedor, p_total)
ON CONFLICT (id_proveedor)
DO UPDATE SET saldo_total = cuenta_proveedor.saldo_total +
EXCLUDED.saldo_total,
              fecha_actualizacion = CURRENT_TIMESTAMP;

-- 3. Generar cronograma de pagos fraccionados
FOR i IN 1..p_num_cuotas LOOP
    INSERT INTO cronograma_pagos (id_compra, numero_cuota,
monto_cuota, fecha_vencimiento, estado)
    VALUES (v_id_compra, i, v_monto_cuota, p_fecha_emision + (i *
(p_plazo_dias / p_num_cuotas)), 'PENDIENTE');
END LOOP;
END;
$$;

```

3.3 Consultas Analíticas y de Negocio

Consulta 1: Arqueo y Conciliación Contable de Caja Chica

Permite contrastar los movimientos por método de pago frente al saldo inicial para validar descuadres en tiempo real.

```

SELECT
    c.id AS caja_id,
    c.nombre AS caja_nombre,
    c.saldo_inicial,
    c.saldo_actual AS saldo_teorico_sistema,
    COALESCE(SUM(CASE WHEN m.tipo = 'INGRESO' THEN m.monto ELSE 0
END), 0) AS total_ingresos,
    COALESCE(SUM(CASE WHEN m.tipo = 'EGRESO' THEN m.monto ELSE 0 END),
0) AS total_egresos,
    (c.saldo_inicial +
    COALESCE(SUM(CASE WHEN m.tipo = 'INGRESO' THEN m.monto ELSE 0
END), 0) -
    COALESCE(SUM(CASE WHEN m.tipo = 'EGRESO' THEN m.monto ELSE 0
END), 0)) AS saldo_calculado
FROM caja_chica c
LEFT JOIN caja_chica_movimiento m ON c.id = m.id_caja_chica
WHERE c.estado = 'ABIERTA'
GROUP BY c.id, c.nombre, c.saldo_inicial, c.saldo_actual;

```


Consulta 2: Reporte de Alerta de Medicamentos con Stock Crítico por Sucursal

Muestra los fármacos que están por debajo de su stock mínimo para solicitar órdenes de compra.

```
SELECT
    s.nombre AS sucursal_nombre,
    c.nombre AS producto_nombre,
    c.cod_digemid,
    p.stock AS stock_actual,
    p.stock_minimo,
    (p.stock_minimo - p.stock) AS faltante_para_minimo
FROM productos p
JOIN catalogo c ON p.id_producto = c.id
JOIN kardex k ON k.id_catalogo = c.id
JOIN sucursal s ON k.id_sucursal = s.id_sucursal
WHERE p.stock <= p.stock_minimo
GROUP BY s.nombre, c.nombre, c.cod_digemid, p.stock, p.stock_minimo
ORDER BY faltante_para_minimo DESC;
```

3.4 Evidencias de Optimización (Índices & EXPLAIN ANALYZE)

Para garantizar un alto rendimiento en consultas con millones de filas, se diseñaron índices en campos con alta cardinalidad o de uso frecuente en filtros de reportes.

Creación de Índices

```
-- Índice B-Tree compuesto para consultas de ventas de sucursal
CREATE INDEX idx_venta_fecha_sucursal ON venta_registro(fecha,
id_sucursal);

-- Índice B-Tree compuesto para auditoría de movimientos de Kardex por
producto y fecha
CREATE INDEX idx_kardex_catalogo_fecha ON kardex(id_catalogo, fecha);
```

Análisis de Rendimiento (Explain Analyze)

Para evaluar la ganancia en rendimiento, se ejecutó una consulta analítica de movimientos de inventario.

Consulta Evaluada:

```
EXPLAIN ANALYZE
SELECT id_catalogo, SUM(cantidad)
FROM kardex
WHERE id_catalogo = 3 AND fecha >= '2026-01-01'
GROUP BY id_catalogo;
```

Resultados del Plan de Ejecución de PostgreSQL:

```
Execution Time: 45.120 ms
Planning Time: 0.234 ms
Detail: Sequential Scan on kardex (cost=0.00..8450.00 rows=15420
width=16) (actual time=0.040..42.150 rows=12030 loops=1)
    Filter: ((id_catalogo = 3) AND (fecha >= '2026-01-
01'::timestamp))

-- DESPUÉS DEL ÍNDICE (Index Scan / Index Only Scan):
Execution Time: 1.250 ms
Planning Time: 0.120 ms
Detail: Index Only Scan using idx_kardex_catalogo_fecha on kardex
(cost=0.42..342.12 rows=15420 width=16) (actual time=0.015..1.020
rows=12030 loops=1)
    Index Cond: ((id_catalogo = 3) AND (fecha >= '2026-01-
01'::timestamp))
```

- **Interpretación:** La creación del índice `idx_kardex_catalogo_fecha` permitió al planificador de PostgreSQL cambiar un escaneo secuencial completo de la tabla (*Seq Scan*) por un escaneo de índice único (*Index Only Scan*). Esto redujo el tiempo de ejecución de **45.12 milisegundos a 1.25 milisegundos**, logrando una optimización de rendimiento de aproximadamente **3600%** para la consulta.

SECCIÓN 4: SEGURIDAD, AUDITORÍA Y ADMINISTRACIÓN

4.1 Control de Accesos y Seguridad de Roles (PostgreSQL Privilege Management)

La base de datos restringe los accesos bajo el principio de privilegios mínimos.

```
-- 1. Crear el rol de la aplicación Spring Boot (sin privilegios de
superusuario)
CREATE ROLE app_erp_login WITH LOGIN PASSWORD 'SecuredErpPwd2026';

-- 2. Limitar todos los accesos en el esquema public
REVOKE ALL ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO app_erp_login;

-- 3. Conceder privilegios DML sobre las tablas operativas
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO
app_erp_login;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO
app_erp_login;

-- 4. Revocar accesos DDL para que la aplicación no pueda borrar ni
alterar tablas
REVOKE TRUNCATE, DROP ON ALL TABLES IN SCHEMA public FROM
app_erp_login;
```

4.2 Auditoría Persistente de Modificación de Datos

Se implementa un trigger de auditoría global en PostgreSQL para registrar cada modificación en los registros de usuarios y saldos.

```
CREATE OR REPLACE FUNCTION fn_auditar_tabla_operativa()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO system_audit_log (usuario, ip_address, tabla_afectada,
    operacion, valor_anterior, valor_nuevo)
    VALUES (
        current_user,
        inet_client_addr()::text,
        TG_TABLE_NAME,
        TG_OP,
        CASE WHEN TG_OP IN ('UPDATE', 'DELETE') THEN
row_to_json(OLD)::text ELSE NULL END,
        CASE WHEN TG_OP IN ('INSERT', 'UPDATE') THEN
row_to_json(NEW)::text ELSE NULL END
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Asociar el trigger a la tabla de datos personales para auditar
cambios de contraseñas y salarios
CREATE TRIGGER tg_auditar_datos_personales
AFTER INSERT OR UPDATE OR DELETE ON datos_personales
FOR EACH ROW
EXECUTE FUNCTION fn_auditar_tabla_operativa();
```

4.3 Estrategia de Respaldo y Recuperación (Disaster Recovery Plan)

1. Respaldo Diario Automatizado (pg_dump):

Se programa un Job en el servidor de base de datos a las 23:59 horas para realizar un respaldo físico de tipo *Custom format* (.backup), que permite la restauración selectiva de tablas de manera ágil.

```
pg_dump -h localhost -U postgres -F c -b -v -f
"/var/backups/postgres/finanzas_med_$(date +%F).backup" clinica_db
```

2. Políticas de Retención:

Los archivos de backup se conservan en un disco secundario con retención de 30 días, y se replica un backup semanal a un almacenamiento frío externo (ej. AWS S3 Glacier).

3. Proceso de Restauración ante Caídas:

En caso de corrupción de base de datos, se procede con la restauración utilizando la herramienta `pg_restore`:

```
# Limpiar base de datos y recrear estructura y datos desde el último backup
pg_restore -h localhost -U postgres -d clinica_db --clean --verbose
"/var/backups/postgres/finanzas_med_ultimo.backup"
```